

SDK / стек

Параметр	Значение
Название приложения	melbet (Google Play); внутри APK app_name = Win / «Салон Win»
Android Gradle Plugin	8.13.2
minSdk	26
targetSdk	36
Kotlin	да, 2.0.21
Web View	нет
Custom Tabs	нет
Рекламные сети	нет
Аналитика	нет
Permissions	android.permission.INTERNET, android.permission.ACCESS_WIFI_STATE, android.permission.ACCESS_NETWORK_STATE, com.doter.seryk.melbet.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.android.vending.CHECK_LICENSE
Libraries	Kotlin 2.0.21, kotlinx-coroutines-core / kotlinx-coroutines-android 1.9.0, OkHttp 4.12.0, Google Material 1.14.0, AndroidX: activity 1.13.0, appcompat 1.7.1, cardview 1.0.0, constraintlayout 2.2.2, coordinatorlayout 1.3.0, core / core-ktx 1.18.0, core-viewtree 1.0.0, cursoradapter 1.0.0, customview 1.2.0, customview-poolingcontainer 1.0.0, drawerlayout 1.1.1, dynamicanimation 1.1.0, emoji2 / emoji2-views-helper 1.3.0, fragment 1.5.4, graphics-shapes 1.0.1, interpolator 1.0.0, lifecycle (livedata / process / runtime / viewmodel / viewmodel-savedstate), loader 1.0.0, navigationevent 1.0.0, profileinstaller 1.4.0, recyclerview 1.4.0, savedstate 1.2.1, startup-runtime 1.1.1, tracing 1.2.0, transition 1.5.0, vectordrawable / vectordrawable-animated 1.1.0, versionedparcelable 1.1.1, viewpager 1.0.0, viewpager2 1.1.0-beta02, viewbinding 8.13.2, annotation-experimental 1.4.1, arch.core-runtime, compose.runtime-annotation 1.9.0; Google Play PairIP: com.pairip.application.Application, com.pairip.licensecheck
Подозрительные домены	нет
SharedPreferences	cfg: ключ rurl — сохранённая ссылка после проверки; win_salon_finance: transactions (история операций салона), daily_goal (цель дня)
Есть ли клоака	да
Подозрительные слова	rurl, workers.dev, status:"ok", filter, license check, melbet, Accept-Language, User-Agent, X-Device-Model

Какие данные собираются

- имя пакета приложения
- модель телефона
- строка браузера телефона

Как собираются

Сразу при открытии стартового экрана, ещё до того как человек успел что-то нажать, приложение само читает из системы Android имя своего пакета, модель телефона и обычную строку браузера (как будто страницу открыли во встроенном браузере телефона). Отдельного окна с вопросом «разрешить?» при этом нет: шаг проходит незаметно, на экране только полоска загрузки на фоне логотипа.

Если раньше проверка уже прошла и ссылка сохранилась в памяти приложения, эти сведения заново не собирают: приложение сразу достаёт сохранённую ссылку и открывает её. Если сохранённой ссылки нет, как раз тогда и читают пакет, модель и строку браузера для тихого запроса в интернет.

К запросу ещё прикладывают пометку языка страницы Accept-Language, но её не берут из настроек телефона: в коде она жёстко прописана как en-US,en;q=0.9. Рекламный номер устройства, версию Android, страну, оператора и часовой пояс для этой проверки приложение не читает.

Куда отправляются

Сведения уходят тихим запросом в интернет на адрес <https://shiny-term-bfbc.romanenkoiinalog.workers.dev?app=...>, где вместо многоточия подставляется имя пакета приложения. Адрес собран из зашифрованных кусочков внутри кода: сначала расшифровывается основа на workers.dev, затем добавляется «?app=», затем имя пакета. Запасных адресов для этой проверки в коде не видно.

Вместе с адресом уходят заголовки: модель телефона (X-Device-Model), строка браузера (User-Agent) и фиксированная пометка языка (Accept-Language). Запрос идёт в фоне со стартового экрана, человек отдельного экрана «проверка» не видит. Если в ответ приходит уже «боевая» ссылка (не на workers.dev), её сохраняют в настройках приложения под именем cfg, ключ url, чтобы при следующем запуске снова не ходить на проверку.

Как фильтруются пользователи

В самом приложении нет списка стран, языков, «белых» или «запрещённых» пользователей и нет проверки на бота или эмулятор. На проверку уходят имя пакета, модель телефона и строка браузера; язык в заголовке всегда один и тот же и из настроек телефона не берётся.

Решение «кому показать внешнюю страницу, а кому обычное приложение» принимает сервер: приложение смотрит только на итоговый адрес после перехода. Если адрес всё ещё содержит workers.dev — считают, что оффер не дали, и открывают обычное приложение. Если итоговый адрес уже другой — считают, что пришла ссылка на оффер, сохраняют её и открывают снаружи.

В коде есть ещё проверка текста ответа на кусок «"status":"ok"», но обе ветки этой проверки ведут в одно и то же место — обычное приложение. То есть реально развилку делает не этот кусок текста, а именно то, остался ли итоговый адрес на workers.dev или сервер увёл запрос на другой сайт.

Что возвращается

После тихого запроса приложение смотрит на итоговый адрес ответа (тот, куда привёл запрос с учётом переходов) и читает тело ответа. Главный признак для решения — итоговый адрес: содержит ли он кусок workers.dev.

Если итоговый адрес всё ещё на workers.dev, приложение считает ответ «белым» и переходит к обычному режиму приложения. Если итоговый адрес уже другой — приложение считает, что сервер отдал ссылку на внешнюю страницу, сохраняет этот адрес и дальше открывает его. При ошибке сети (запрос не удался) тоже показывают обычное приложение, без внешней страницы.

Отдельного поля вроде offer или url в разборе ответа нет: роль «ссылки на оффер» играет сам итоговый адрес запроса после переходов. Кусок «"status":"ok"» в теле ответа в коде ищут, но на поведение это не влияет — обе ветки одинаковые.

Как показывается оффер или белая версия

Если пришла «боевая» ссылка (итоговый адрес уже не на workers.dev), её открывают во внешнем браузере телефона обычным системным переходом на сайт. Внутри приложения встроенного окна сайта для оффера нет: со стартового экрана с полоской загрузки сразу уводят наружу. То же самое делают, если ссылка уже была сохранена с прошлого раза.

Если ответ «белый», сеть упала или внешний браузер не смог открыть ссылку, человеку просто показывают обычное приложение. Про белую оболочку в коде видно, что это учёт доходов и расходов салона под названием «Салон Win», без перехода на внешнюю рекламную страницу.